

Развёртывание Node.js REST API на Linux-сервере

Ubuntu Server → Node.js → PostgreSQL → Prisma → systemd → Nginx → HTTPS

Цели лекции

- Понять архитектуру ручного deployment Node.js REST API на VPS/Linux.
- Подготовить Ubuntu Server к размещению backend-приложения.
- Установить Node.js 22 и PostgreSQL.
- Развернуть Internet Shop API и применить Prisma migrations.
- Запустить приложение как systemd service.
- Настроить Nginx как reverse proxy.
- Ограничить сетевой доступ через UFW.
- При наличии домена включить HTTPS.
- Научиться диагностировать deployment через systemctl, journalctl и Nginx logs.

Render и собственный Linux-сервер

Render / PaaS

- Платформа скрывает ОС.
- Build/start задаются в Dashboard.
- HTTPS и URL выдаются автоматически.
- Logs доступны через интерфейс.
- Меньше системного администрирования.

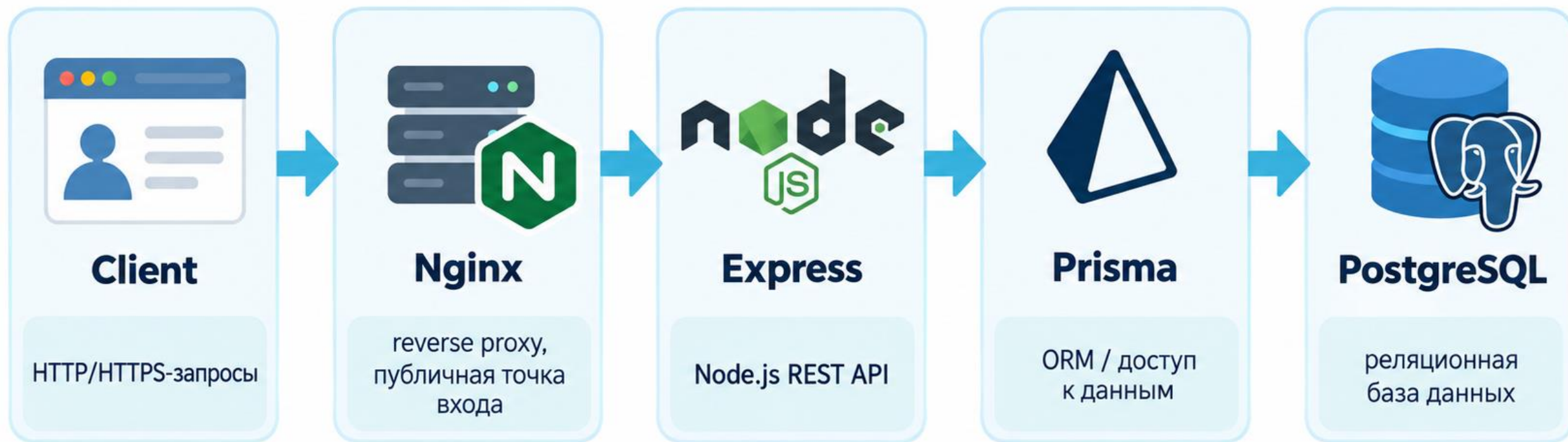
Linux VPS

- Мы администрируем ОС.
- Сами устанавливаем Node.js и PostgreSQL.
- Сами настраиваем systemd и Nginx.
- Сами открываем firewall ports.
- Сами отвечаем за HTTPS и обновления.

1. Архитектура Node.js REST API на Linux-сервере

Какие компоненты участвуют в обработке HTTP-запроса

Production-архитектура



Внешний клиент обращается к Nginx, а приложение работает с базой через Prisma.

Роли компонентов

- Ubuntu Server — операционная система сервера.
- Nginx — принимает внешние HTTP/HTTPS-запросы.
- Node.js / Express — выполняет REST API.
- systemd — запускает и контролирует Node.js process.
- Prisma — работает с данными приложения.
- PostgreSQL — хранит данные Internet Shop.
- UFW — ограничивает доступ к сетевым портам.

Какие порты нужны

Компонент	Порт	Доступ
SSH	22/tcp	Из интернета для администрирования
Nginx HTTP	80/tcp	Из интернета
Nginx HTTPS	443/tcp	Из интернета
Express	3000/tcp	Только localhost
PostgreSQL	5432/tcp	Только localhost

Почему Express не публикуем напрямую

- Node.js process не обязан напрямую принимать внешний интернет-трафик.
- Nginx принимает HTTP/HTTPS-запросы и передаёт их во внутренний Express process.
- Port 3000 остаётся доступным только локально на сервере.
- Так проще настроить domain, HTTPS, proxy headers и единый входной endpoint.
- Внешний клиент работает с 80/443, а не с внутренним портом приложения.

Внешние и внутренние порты



Целевая файловая структура

TEXT

```
/var/www/internet-shop/  
├─ server.js  
├─ app.js  
├─ package.json  
├─ package-lock.json  
├─ prisma/  
├─ src/  
└─ generated/
```

Что важно

- Исходный код находится в отдельном каталоге приложения.
- Secrets не хранятся внутри Git repository.
- Node.js process работает не от root.



2. Подготовка Ubuntu Server

SSH, updates, отдельный пользователь и базовая безопасность

Учебная серверная среда

- Ubuntu Server 26.04 LTS.
- Публичный IPv4-адрес VPS или виртуальной машины.
- SSH-доступ с `sudo privileges`.
- Доступ в интернет для установки `packages`.
- Git repository с Internet Shop API.

Почему используем LTS и Node.js 22

- LTS-релизы операционной системы и Node.js рассчитаны на длительную поддержку.
- В учебном проекте используется Node.js 22, поэтому production-сервер должен использовать совместимую major-version.
- Одинаковая major-version снижает риск различий между локальной разработкой, CI и сервером.
- Если проект позже переводится на новую LTS-версию Node.js, это нужно делать осознанно и после тестирования.
- Главное правило: production-среда должна быть предсказуемой и поддерживаемой.

Первое подключение по SSH

BASH

```
ssh ubuntu@SERVER_IP
```

Обновление packages

BASH

```
sudo apt update  
sudo apt upgrade -y
```

Что важно

- apt update обновляет metadata package repositories.
- apt upgrade устанавливает доступные обновления.
- Production server должен регулярно получать security updates.

Создаём отдельного пользователя приложения

BASH

```
sudo adduser shop  
sudo usermod -aG sudo shop
```

Что важно

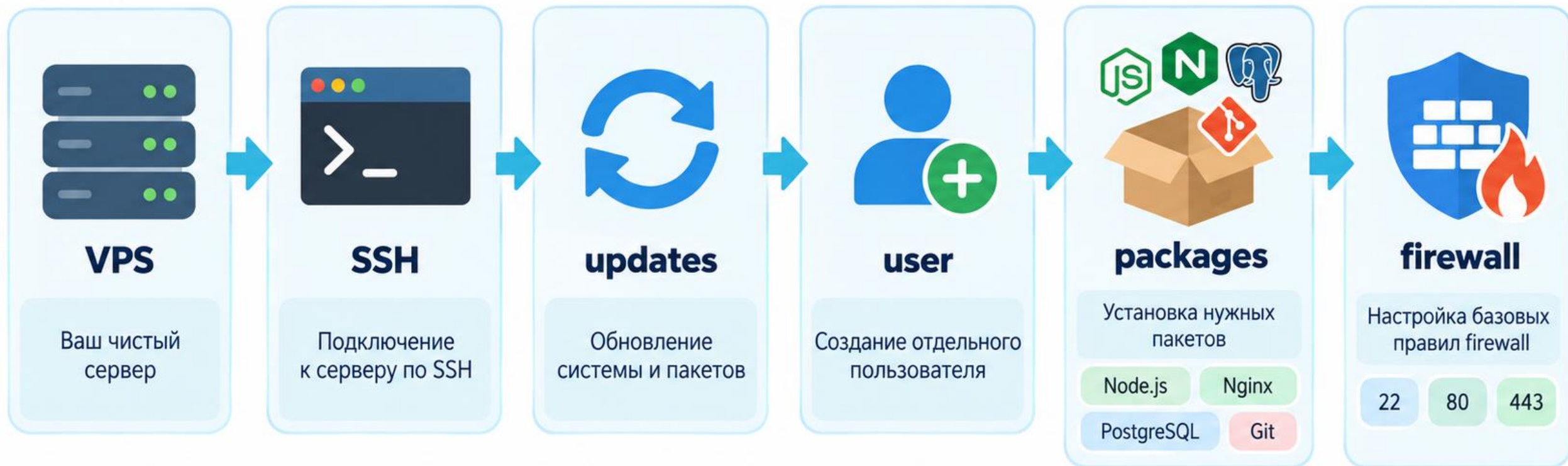
- shop будет владельцем файлов Internet Shop.
- Не запускаем Node.js от root.
- sudo можно оставить для учебной лаборатории; в production privileges обычно ограничивают сильнее.

Базовые инструменты

BASH

```
sudo apt install -y git curl ca-certificates nginx postgresql postgresql-contrib
```

Подготовка VPS



Результат: готовая и безопасная VPS, на которой можно развернуть Node.js REST API.

Вы получаете стабильное окружение, отдельного пользователя, необходимые пакеты и базовую защиту.

3. Установка Node.js 22 и подготовка проекта

Системный Node.js, Git clone, npm ci и Prisma Client

Подключение NodeSource repository для Node.js 22

BASH

```
curl -fsSL https://deb.nodesource.com/setup_22.x -o nodesource_setup.sh  
sudo -E bash nodesource_setup.sh  
sudo apt install -y nodejs
```

Проверка Node.js

BASH

```
node --version  
npm --version  
which node
```

Каталог приложения

BASH

```
sudo mkdir -p /var/www/internet-shop  
sudo chown -R shop:shop /var/www/internet-shop
```

Что важно

- Создаём отдельный каталог приложения.
- Владельцем назначаем пользователя shop.
- systemd затем будет запускать процесс от этого же пользователя.

Получаем проект из Git

BASH

```
sudo -iu shop  
cd /var/www/internet-shop  
git clone https://github.com/USER/internet-shop-api.git .
```

Установка dependencies

BASH

```
cd /var/www/internet-shop  
npm ci
```

Что важно

- npm ci использует package-lock.json.
- Команда подходит для воспроизводимой установки.
- После чистого clone node_modules создаётся заново на сервере.

Что не переносим с Windows на Linux-сервер

- Не копируем `node_modules`: зависимости устанавливаются на сервере через `npm ci`.
- Не копируем локальный `.env`: `production variables` хранятся отдельно.
- Не копируем локальную PostgreSQL database: на сервере создаётся собственная база.
- Не копируем временные `logs`, `coverage`, `cache` и IDE-файлы.
- На сервер переносится исходный код, `package files`, `Prisma schema` и `migrations`.



4. PostgreSQL и Prisma на сервере

Создание production database, пользователя и применение migrations

Проверяем PostgreSQL

BASH

```
sudo systemctl status postgresql  
sudo -u postgres psql
```

Создаём database user и database

SQL

```
CREATE USER shop_user WITH PASSWORD 'CHANGE_ME_STRONG_PASSWORD';  
CREATE DATABASE shop_db OWNER shop_user;  
\q
```

Production DATABASE_URL

TEXT

```
postgresql://shop_user:STRONG_PASSWORD@127.0.0.1:5432/shop_db?schema=public
```

Что важно

- DATABASE_URL на Linux-сервере указывает на PostgreSQL, работающую на этом же сервере.
- Используем 127.0.0.1, а не публичный IP.
- PostgreSQL не требуется открывать в интернет.
- Пароль database user хранится вне Git repository.

Почему production variables хранятся отдельно

- Production secrets не должны попадать в Git repository.
- DATABASE_URL может содержать пароль database user.
- JWT_SECRET должен быть отдельным значением для production.
- Файл /etc/internet-shop.env подключается systemd service через EnvironmentFile.
- Доступ к файлу ограничивается permissions, чтобы его не могли читать посторонние пользователи.

Environment file

BASH

```
sudo nano /etc/internet-shop.env
```

Содержимое /etc/internet-shop.env

TEXT

```
NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://shop_user:STRONG_PASSWORD@127.0.0.1:5432/shop_db?schema=public
JWT_SECRET=CHANGE_ME_TO_RANDOM_SECRET
CORS_ORIGIN=https://example.com
```

Ограничиваем доступ к environment file

BASH

```
sudo chown root:shop /etc/internet-shop.env  
sudo chmod 640 /etc/internet-shop.env
```

Что важно

- root владеет файлом.
- Группа shop может прочитать значения.
- Другие пользователи не получают доступ.
- Secrets не должны выводиться в отчёт или Git.

Prisma Client и migrations

BASH

```
cd /var/www/internet-shop  
npx prisma generate  
npx prisma migrate deploy
```

migrate deploy в Production

- Применяет pending migrations.
- Не создаёт новые migrations.
- Не использует shadow database.
- Не сбрасывает production database.
- Не заменяет prisma generate.

Подготовка БД

От установки PostgreSQL до готовой схемы Prisma



PostgreSQL



Prisma

Next-generation ORM



1

PostgreSQL

Установка и настройка



```
# Установка PostgreSQL
sudo apt update
sudo apt install postgresql
postgresql-contrib

# Проверка статуса
sudo systemctl status postgresql

# Создание БД и пользователя
sudo -u postgres psql
CREATE USER appuser
WITH PASSWORD 'strong_password';
CREATE DATABASE appdb
OWNER appuser;
\q
```

PostgreSQL установлен
и база данных создана.

2

DATABASE_URL

Настройка строки
подключения



```
# .env (production)
DATABASE_URL="postgresql://
appuser:strong_password@
localhost:5432/appdb?
schema=public"
```

Строка подключения
хранится в переменных
окружения (.env)
и используется Prisma.

3

prisma generate

Генерация клиента



Prisma

```
# Генерация Prisma Client
npx prisma generate
```

- ✓ Environment variables loaded from .env
- ✓ Prisma schema loaded from prisma/schema.prisma
- ✓ Generated Prisma Client to ./node_modules/@prisma/client

Создаётся Prisma Client
для работы с базой данных
в приложении.

4

prisma migrate deploy

Применение миграций



```
# Применение миграций на сервере
npx prisma migrate deploy
```

- ✓ Environment variables loaded from .env
- ✓ Applying migrations...
- ✓ All migrations have been applied successfully.

Миграции применяются
к базе данных на сервере.
Схема БД обновляется
до актуального состояния.

5

Готовая schema

База данных готова
к работе



Таблицы в базе данных

- users
- posts
- comments
- categories
- _prisma_migrations
- ...

Структура базы данных
создана и готова
к использованию
в приложении.



База данных настроена и готова к работе!

PostgreSQL + Prisma = Надёжная основа вашего API

5. **systemd**: постоянный запуск Node.js

Автозапуск, restart и централизованные logs

Почему недостаточно `node server.js`

- SSH session может завершиться.
- После reboot процесс не запустится автоматически.
- При crash приложение останется недоступным.
- Нужен системный менеджер долгоживущего процесса.
- В Linux эту задачу выполняет `systemd`.

systemd управляет Node.js

1 boot



Сервер
загружается

2 start



systemd



systemd
запускает сервис

3 running



Приложение
работает

4 crash



Процесс завершился
с ошибкой

5 restart



systemd



systemd
перезапускает сервис

systemd следит за процессом
и автоматически восстанавливает его



systemd постоянно следит за состоянием сервиса **Node.js**.

Если процесс завершается с ошибкой, **systemd** автоматически перезапускает его, чтобы приложение снова работало.

Создаём unit file

BASH

```
sudo nano /etc/systemd/system/internet-shop.service
```

internet-shop.service

INI

[Unit]

Description=Internet Shop REST API

After=network.target postgresql.service

[Service]

Type=simple

User=shop

Group=shop

WorkingDirectory=/var/www/internet-shop

EnvironmentFile=/etc/internet-shop.env

ExecStart=/usr/bin/node /var/www/internet-shop/server.js

Restart=on-failure

RestartSec=5

[Install]

WantedBy=multi-user.target

Разбираем unit file

INI

```
User=shop
WorkingDirectory=/var/www/internet-shop
EnvironmentFile=/etc/internet-shop.env
ExecStart=/usr/bin/node /var/www/internet-shop/server.js
Restart=on-failure
```

Что важно

- User — процесс не работает от root.
- WorkingDirectory — рабочий каталог проекта.
- EnvironmentFile — production configuration.
- ExecStart использует абсолютный путь к Node.js и server.js.
- Restart=on-failure перезапускает сервис после аварийного завершения.

Активируем service

BASH

```
sudo systemctl daemon-reload  
sudo systemctl enable --now internet-shop  
sudo systemctl status internet-shop
```

Logs через journalctl

BASH

```
sudo journalctl -u internet-shop -n 100 --no-pager  
sudo journalctl -u internet-shop -f
```

Проверяем API локально на сервере

BASH

```
curl http://127.0.0.1:3000/health/live  
curl http://127.0.0.1:3000/health/ready
```

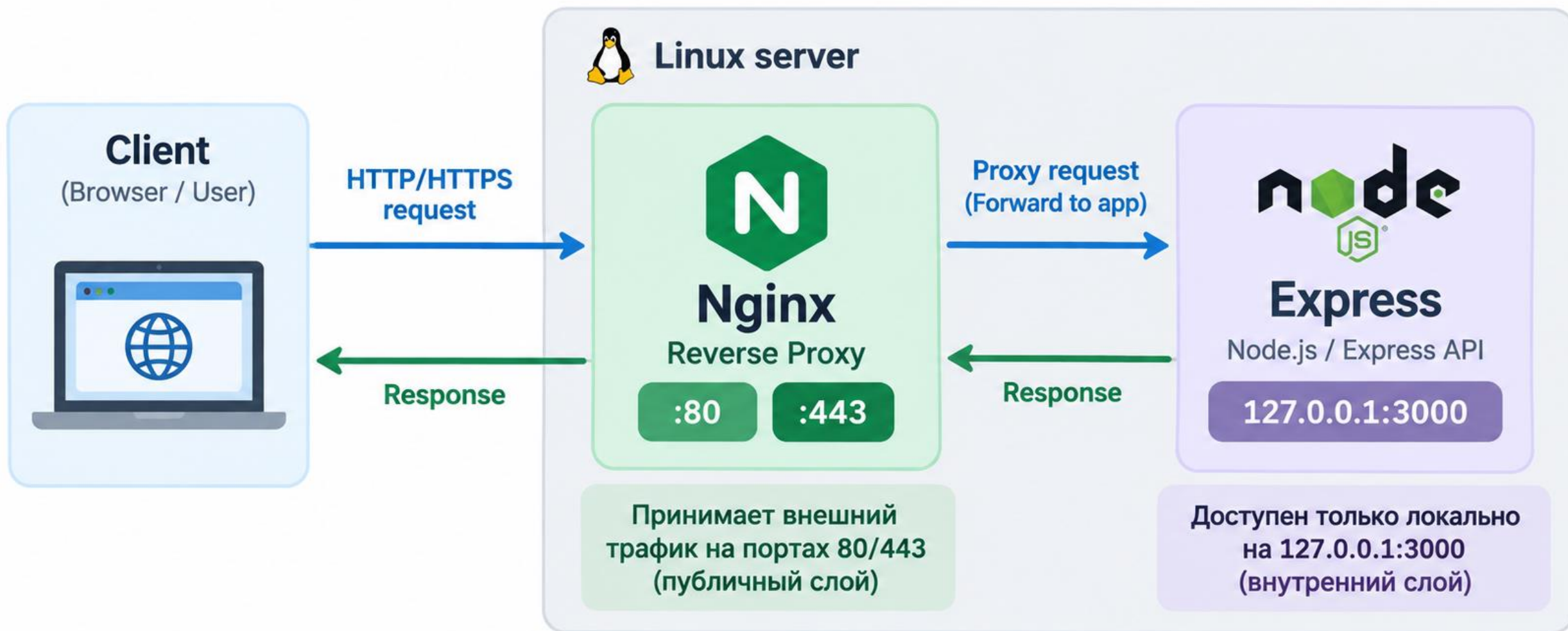
Промежуточная контрольная точка перед Nginx

- Перед настройкой Nginx нужно убедиться, что само Node.js-приложение уже работает.
- `systemctl status internet-shop` должен показывать active (running).
- `journalctl` не должен содержать startup errors.
- `curl http://127.0.0.1:3000/health/live` должен вернуть успешный ответ.
- `curl http://127.0.0.1:3000/health/ready` должен подтвердить доступность PostgreSQL.
- Если эта проверка не пройдена, Nginx пока не настраиваем: сначала исправляем приложение или database connection.

6. Nginx как reverse proxy

Публичный HTTP/HTTPS endpoint перед Express

Reverse proxy – Client → Nginx :80/:443 → Express 127.0.0.1:3000



Nginx публикуется наружу, Express не открывается напрямую в интернет.

Весь внешний трафик проходит через Nginx, а Express доступен только локально на сервере.

Что делает Nginx

- Принимает входящие HTTP/HTTPS connections.
- Перенаправляет запросы в Express через `proxy_pass`.
- Передаёт Host и client IP headers.
- Скрывает внутренний порт Node.js.
- Может обслуживать TLS certificate.

Nginx configuration

NGINX

```
server {  
    listen 80;  
    server_name api.example.com;  
  
    location / {  
        proxy_pass http://127.0.0.1:3000;  
        proxy_http_version 1.1;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

Включаем конфигурацию Nginx

BASH

```
sudo nano /etc/nginx/sites-available/internet-shop  
sudo ln -s /etc/nginx/sites-available/internet-shop /etc/nginx/sites-enabled/internet-shop  
sudo nginx -t  
sudo systemctl reload nginx
```

trust proxy в Express

- После reverse proxy исходный запрос приходит в Express от Nginx.
- X-Forwarded-* headers содержат информацию об исходном запросе.
- Если приложение использует req.ip, req.protocol или secure cookies, нужно корректно настроить trust proxy.
- Нельзя бездумно доверять произвольным proxy headers из недоверенной сети.

Пример trust proxy для Nginx на том же сервере

JAVASCRIPT

```
app.set('trust proxy', 'loopback');
```

Проверяем Nginx

BASH

```
curl http://SERVER_IP/health/live  
curl http://SERVER_IP/api/products
```



7. Firewall и HTTPS

Открываем только необходимые порты и защищаем внешний трафик

UFW: минимальный набор правил

BASH

```
sudo ufw allow OpenSSH  
sudo ufw allow 'Nginx Full'  
sudo ufw enable  
sudo ufw status
```

Почему нельзя открывать 3000 и 5432 в интернет

- 3000/tcp — внутренний порт Express; внешний доступ должен идти через Nginx.
- 5432/tcp — порт PostgreSQL; database не должна быть доступна из интернета.
- Открытый PostgreSQL увеличивает риск перебора паролей и атак на базу данных.
- Открытый Express port создаёт второй внешний вход в приложение в обход Nginx.
- Минимальный внешний набор портов: SSH 22, HTTP 80 и HTTPS 443.

HTTPS при наличии домена

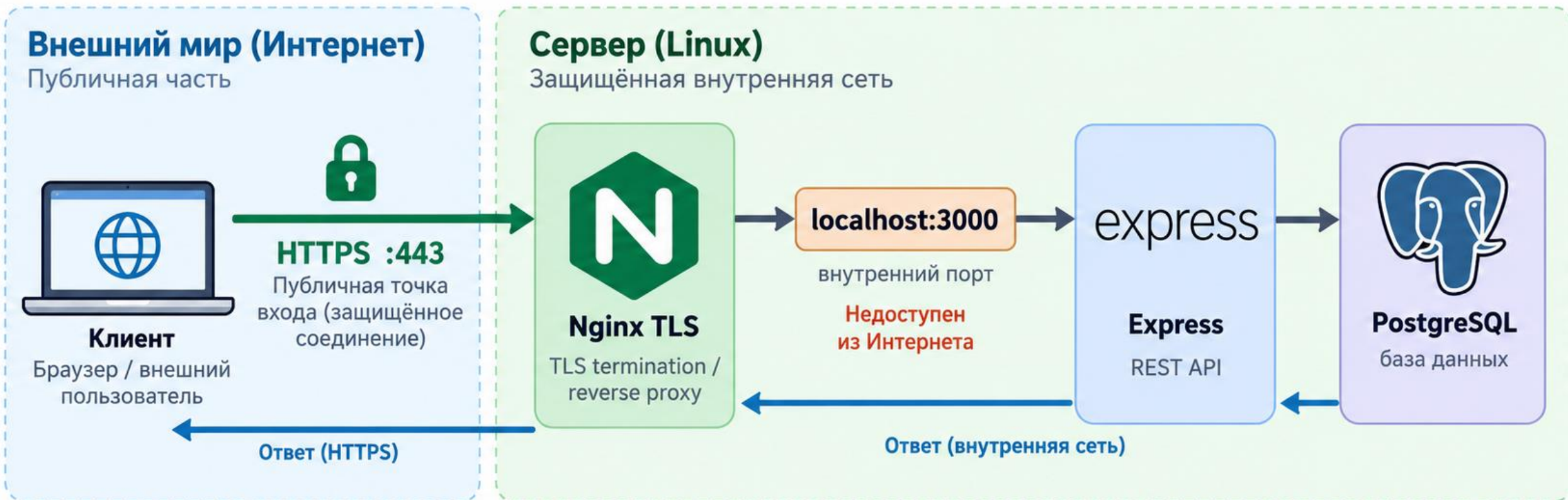
- DNS A-record должен указывать на public IP сервера.
- Nginx должен отвечать на HTTP для домена.
- Certbot может получить certificate Let's Encrypt и настроить Nginx.
- После настройки клиент использует `https://api.example.com`.
- Для лаборатории без домена HTTPS можно оставить обзорным этапом.

Certbot для Nginx

BASH

```
sudo snap install --classic certbot  
sudo ln -s /snap/bin/certbot /usr/local/bin/certbot  
sudo certbot --nginx -d api.example.com
```

Защищённый путь запроса



Только HTTPS :443 доступен из Интернета. Все остальные сервисы (localhost:3000, PostgreSQL) находятся во внутренней сети сервера и не доступны извне.



Внутренние соединения (localhost:3000, PostgreSQL) доступны только локально на сервере (127.0.0.1) и не имеют публичного доступа.

8. Диагностика Linux deployment

Где искать ошибку, если API недоступен

Диагностика по уровням

Симптом	Уровень	Команда
SSH недоступен	Network/firewall	ssh, ufw status
502 Bad Gateway	Nginx → Node.js	systemctl status internet-shop
API process failed	systemd/Node.js	journalctl -u internet-shop
Database error	PostgreSQL/Prisma	systemctl status postgresql
Nginx config error	Nginx	nginx -t
HTTPS error	DNS/Certbot/Nginx	certbot certificates, nginx logs

Алгоритм диагностики

- 1 Process** Проверить `systemctl status internet-shop`.
- 2 Application logs** Открыть `journalctl` и найти первую ошибку.
- 3 Local API** Проверить `curl 127.0.0.1:3000`.
- 4 Nginx** Проверить `nginx -t` и `curl` через port 80/443.
- 5 Database** Проверить PostgreSQL и readiness endpoint.
- 6 Network** Проверить firewall, DNS и внешний URL.

Диагностика Linux deployment

Пошаговая проверка всего пути от сервиса до публичного API



Быстро находим и
устраняем проблемы



Сервисы работают



Порты открыты



Доступен из интернета

1

systemd

Проверяем статус сервиса
Node.js



node-api.service

```
sudo systemctl status  
node-api.service
```

● active (running)

```
Loaded: loaded  
Active: active (running)  
Main PID: 1234  
...
```

Сервис запущен
и управляется systemd.

2

journalctl

Смотрим логи сервиса



```
sudo journalctl -u  
node-api.service -f
```

```
Sep 21 10:15:01 server  
node-api[1234]: Server  
started on port 3000  
Sep 21 10:15:02 server  
node-api[1234]:  
Connected to database  
...
```

В логах нет ошибок.
Приложение стартовало
корректно.

3

localhost:3000

Проверяем доступность
приложения локально



127.0.0.1:3000

```
curl http://localhost:3000/  
HTTP/1.1 200 OK  
Content-Type: application/json  
{ "status": "ok" }
```

Приложение отвечает
на локальном порту 3000.

4

Nginx

Проверяем конфигурацию
и проксирование



/etc/nginx/nginx.conf

```
sudo nginx -t  
# syntax is ok  
# configuration is ok  
sudo systemctl status nginx  
# active (running)  
curl -I http://localhost  
HTTP/1.1 301 Moved Permanently  
Location: https://example.com
```

Nginx работает
и корректно проксирует
запросы к Express.

5

Firewall / DNS

Проверяем открытые порты
и DNS-имя



UFW / DNS

```
sudo ufw status  
Status: active  
To Action  
22/tcp ALLOW  
80/tcp ALLOW  
443/tcp ALLOW  
3000/tcp DENY  
5432/tcp DENY  
nslookup example.com  
# Address: 203.0.113.10
```

Открыты только нужные
порты. DNS-имя корректно
разрешается.

6

public API

Проверяем доступность
из интернета



https://api.example.com

```
curl -I https://api.example.com  
HTTP/1.1 200 OK  
Server: nginx  
Content-Type: application/json  
curl https://api.example.com/  
{ "status": "ok" }
```

API доступен из интернета
и работает корректно.



Весь путь работает!

От сервиса до публичного API



Быстрая диагностика



Понятные команды



Надёжная работа



Проверяйте поэтапно —
это помогает быстро найти
и устранить проблему.

Практическая задача

Развернуть Internet Shop REST API на Ubuntu Server так, чтобы приложение автоматически запускалось и было доступно через Nginx.

Задание

- Установить Node.js 22, PostgreSQL и Nginx.
- Создать database и database user.
- Склонировать Internet Shop.
- Создать production environment file.
- Выполнить npm ci, prisma generate и prisma migrate deploy.
- Создать и запустить systemd service.
- Настроить Nginx reverse proxy.
- Включить UFW.
- Проверить public REST endpoints.

Критерии проверки

- systemd service active.
- Express не доступен напрямую через public :3000.
- Nginx проксирует запросы в API.
- PostgreSQL доступна приложению.
- /health/ready возвращает 200.
- После reboot service запускается автоматически.

Проверка понимания

- 1 Почему Node.js process не следует запускать от root?
- 2 Зачем нужен systemd?
- 3 Почему PostgreSQL и Express оставляем на localhost?
- 4 Для чего используется Nginx reverse proxy?
- 5 Что делает prisma migrate deploy?
- 6 Где хранятся production environment variables?
- 7 Почему сначала проверяем localhost:3000, а затем Nginx?
- 8 Какие внешние порты должен разрешать firewall?

Итоги лекции

1

Linux deployment даёт полный контроль над инфраструктурой и требует ручной настройки.

2

Node.js REST API запускается отдельным пользователем.

3

PostgreSQL работает локально и не публикуется в интернет.

4

Prisma Client генерируется отдельно, production migrations применяются через migrate deploy.

5

systemd обеспечивает автозапуск, restart и централизованные logs.

6

Nginx публикует REST API через HTTP/HTTPS и проксирует запросы в Express.

7

UFW открывает только необходимые внешние порты.

8

HTTPS добавляется через Nginx и certificate при наличии домена.

9

Диагностику выполняем последовательно: process → logs → local API → proxy → network.

10

Следующая тема: обновление и сопровождение REST API в Production.